BITÁCORA DE LABORATORIO S08

Sistemas Distribuidos · Ciclo 2027-1 · Grupo 2

Estudiante: Muñoz Victoria
Fecha de entrega: 17 de octubre de 2026

Tema: Saga orquestada, compensación de transferencias y circuit breaker

Documento de evidencia textual. No incluye imágenes ni capturas.

1. Objetivo

Implementar y validar una saga orquestada para la transferencia entre cuentas de Mexi Banco. La solución debe manejar el fallo del abono después de realizar el cargo, intentar devolver el dinero a la cuenta origen y registrar la compensación. Además, las llamadas desde el servicio de transferencia al servicio de cuentas deben estar protegidas mediante un circuit breaker.

2. Entorno de trabajo

- Repositorio de trabajo de la práctica: mexi-banco-s08.
- Aplicación de transferencia desarrollada con Java y Spring Boot.
- Ejecución local y de servicios mediante Docker Compose.
- Pruebas automatizadas ejecutadas con Maven Wrapper (./mvnw test).
- Circuit breaker configurado mediante Resilience4j.

3. Parte 0: reproducción del fallo distribuido

Se reprodujo un escenario en el que el cargo en la cuenta origen se confirma, pero el servicio de cuentas deja de estar disponible antes de completar el abono en la cuenta destino. La petición de transferencia terminó con una respuesta HTTP 504. Al consultar los saldos, la cuenta origen había disminuido en $200 y la cuenta destino conservaba su saldo inicial, lo que evidencia que una transacción local no garantiza la atomicidad de una operación distribuida entre servicios.

Datos observados:

- Cuenta origen: 002180001710000005. Saldo inicial reportado: $1,000; saldo después del fallo: $800.
- Cuenta destino: 002180001710000006. Saldo inicial reportado: $500; saldo después del fallo: $500.
- Monto de la transferencia: $200.
- Se reinició el servicio de cuentas y las réplicas volvieron a estar saludables.

4. Parte 1: saga orquestada y compensación

Se modificó el servicio TransferenciaService para registrar la transferencia y mantener su estado durante el flujo. Antes de realizar el cargo, se consultan las cuentas y se guarda la transferencia en estado EN_PROCESO. Si falla el cargo, se marca como FALLIDA. Después del cargo se intenta realizar el abono.

Estados implementados

- **EN_PROCESO:** la transferencia está en ejecución.
- **COMPLETADA:** el cargo y el abono finalizaron correctamente.
- **COMPENSADA:** falló el abono y se logró reembolsar el monto a la cuenta origen, registrando el movimiento de reembolso.
- **COMPENSACION_PENDIENTE:** no se pudo completar el proceso de compensación; requiere revisión y recuperación controlada.
- **FALLIDA:** no se logró ejecutar el cargo.

Para la compensación se utiliza el tipo de movimiento TRANSFERENCIA_REEMBOLSADA. Si falla el abono de destino, el sistema intenta abonar el monto a la cuenta origen y registrar el movimiento correspondiente. Si la compensación no se puede completar, la transferencia queda en COMPENSACION_PENDIENTE.

Consideración de recuperación: antes de reintentar una compensación pendiente debe verificarse si el abono de reembolso ya se ejecutó. Sin una clave de idempotencia o una verificación equivalente, un reintento ciego podría duplicar el reembolso.

5. Parte 2: circuit breaker

Se integró Resilience4j en CuentaCliente para proteger las llamadas de consulta, cargo y abono al servicio de cuentas. La configuración utiliza una ventana basada en cuatro llamadas, un mínimo de tres llamadas para calcular la tasa de fallos, un umbral de fallos del 50 % y un periodo de espera de 10 segundos antes de permitir el estado HALF_OPEN.

Estados observados durante la validación

- **CLOSED:** las llamadas se permiten normalmente.
- **OPEN:** tras registrar fallos suficientes, el circuito se abre y rechaza llamadas protegidas.
- **HALF_OPEN:** después del periodo de espera se permiten llamadas de prueba para comprobar recuperación.
- **Recuperación confirmada:** después de una transferencia exitosa, el endpoint de Actuator reportó el circuito en CLOSED, con dos llamadas registradas en la ventana y cero fallos.

La consulta directa a una cuenta a través del gateway no valida el circuit breaker de CuentaCliente, porque esa ruta evita el cliente protegido. Para validar el cierre se realizó una transferencia exitosa mediante el servicio de transferencias.

6. Pruebas automatizadas

Se ejecutó el comando `./mvnw test` desde el directorio transferencia. Maven reportó BUILD SUCCESS.

- BalanceoPorNombreTest: 1 prueba, sin fallos.
- ClientesHttpTest: 3 pruebas, sin fallos.
- TransferenciaServiceTest: 5 pruebas, sin fallos.
- Total: 9 pruebas; 0 fallos, 0 errores y 0 omitidas.

7. Prueba de transferencia exitosa

Se envió una transferencia de prueba de $1 entre las cuentas 0021800000000000001 y 0021800000000000002. La API devolvió HTTP 201 Created y el registro reportó el identificador 7 con estado COMPLETADA. Posteriormente, el endpoint `/actuator/circuitbreakers` reportó el circuito `cuenta` en estado CLOSED.

Esta prueba modificó los saldos de las cuentas del entorno de laboratorio; el documento no afirma que los saldos hayan sido restaurados.

8. Archivos y cambios principales

- transferencia/src/main/java/mx/mexibanco/transferencia/TransferenciaService.java: orquestación y compensación.
- transferencia/src/main/java/mx/mexibanco/transferencia/EstadoTransferencia.java: estados de la saga.
- transferencia/src/main/java/mx/mexibanco/transferencia/Transferencia.java: persistencia del estado.
- transferencia/src/main/java/mx/mexibanco/clientes/CuentaCliente.java: protección con Resilience4j.
- transferencia/src/main/resources/application.yml: configuración del circuit breaker y Actuator.
- transferencia/src/test/java/mx/mexibanco/transferencia/TransferenciaServiceTest.java: pruebas de escenarios de transferencia.

- movimiento/src/main/java/mx/mexibanco/movimiento/TipoMovimiento.java: tipo de movimiento de reembolso.

9. Conclusiones y limitaciones

La práctica permitió observar el problema de consistencia que aparece cuando una operación cruza límites de servicios y confirmar que una saga puede modelar una compensación de negocio. También se validó que el circuit breaker cambia de estado ante fallos y permite comprobar la recuperación cuando el servicio vuelve a responder.

La compensación distribuida no equivale a una transacción atómica. En particular, el estado COMPENSACION_PENDIENTE necesita un mecanismo operacional de recuperación y una estrategia de idempotencia para evitar efectos duplicados. Las pruebas unitarias y las verificaciones descritas no sustituyen pruebas adicionales de fallos durante cada paso de la compensación.

Esta bitácora se redactó con los resultados reportados durante la sesión de trabajo. No incluye capturas de pantalla ni evidencia visual, conforme a la solicitud.